
When Adversarial Prediction Still Compresses: Tokenization and Adversarial Robustness in LLM-Based Lossless Compression

Anonymous Author(s)

Affiliation

Address

email

Abstract

1 LLM token tables can act as global string compressors: variable-length byte
2 sequences become fixed-width token IDs that support fast decoding and execution
3 on encoded data. Predictive LLM compression adds a second layer, entropy-coding
4 those IDs with next-token probabilities. We show that robustness depends on the
5 product of both layers—tokens per source byte and coded bits per token—and
6 cannot be inferred from either token-table compression ratio or perplexity alone.
7 We construct canonical adversarial sequences and separately attribute tokenization
8 gain, model surprise, arithmetic-coder loss, dictionary metadata, and framing.
9 For Qwen2.5-0.5B, full-vocabulary adversarial sequences cost 5.73 times more
10 bits/token than matched text8, but tokenization absorbs enough of that cost to retain
11 a 51.3% ideal byte rate. When restricted to one-byte tokens, however, the same
12 predictive layer expands data to a 143.4% arithmetic payload. Post-hoc vocabulary
13 masking reduces adversarial model cost by 35.2%. These results identify tokenizer
14 fertility as a robustness boundary and motivate co-designing token tables, predictive
15 coders, and guarded fallback paths.

16 1 Who is actually doing the heavy lifting in LLM-based compression?

17 “Language Modeling Is Compression” introduced and formalized how next-symbol probabilities can
18 drive a lossless arithmetic coder and describes tokenization itself as lossless precompression [Delétang
19 et al., 2024]. This observation implies that so-called LLM compression combines two distinct
20 mechanisms: a tokenizer replaces variable-length source strings with token IDs, and a language
21 model predicts those IDs so that an entropy coder can shorten likely ones. That the tokenization itself
22 already provides some compressive benefit is often under-stated and has not been systematically,
23 jointly with LLM-based entropy compression to detangle both compressive effects studied.

24 Schmidt et al. [?] isolate the first mechanism before introducing a They use an LLM vocabulary
25 as a global symbol table in which common byte strings map to fixed-width IDs and one-byte entries
26 guarantee coverage. Unlike a block-local code, this stable representation supports equality checks,
27 hashing, joins, and aggregations before decoding. These properties make token tables particularly
28 attractive for databases, where strings can remain encoded during common relational operations and
29 be decoded only when needed. Thus tokenization can be a useful compression primitive without
30 language-model inference. What remains unclear is how much of an LLM codec’s behavior comes
31 from this token representation and how much comes from prediction. Delétang et al. compare ASCII
32 and BPE [Sennrich et al., 2016] on natural enwik9, but the two effects have not been isolated under
33 inputs chosen against realized end-to-end size. This matters because prediction loss is measured per
34 token while compression is measured per source byte: a poor probability model may still sit inside a

pipeline that compresses when its tokens represent many bytes, whereas low-fertility tokenization can expose expansion. We therefore study the tokenizer–predictor–coder pipeline as a two-layer system.

Contributions. We (i) factor end-to-end compression rate into tokenizer fertility and predictive coding under natural text, random input and worst-case inputs; (ii) we formalize token-, byte-, and realized-code objectives and evaluate against worst-case attacks per objective to demonstrate when the token layer absorbs predictive surprise and when it exposes expansion of the original file size and (iii) we propose a systems design for database compression that is LLM-native while still having the advantages of dictionary encodings (querability, ...).

Systems implication. The two-layer (tokenization + LLM-based entropy coding) view suggests a robust architecture for learned compression. Token IDs (global token table for all tables) can remain the stable, queryable representation used during processing, while predictive entropy coding exploiting LLM-next token probabilities is applied only as an optional storage, transport layer or to actually cold data. This separates the operational benefits of a global token representation from the inference cost and ... of the language model and introduces tokenization as LLM-native pre-compression.

2 Compression objectives and how to construct worst-case inputs for the predictor

Our central distinction is between prediction difficulty and end-to-end compression failure. A token can be extremely unlikely under the language model while decoding to many source bytes, so maximizing token surprisal need not maximize compressed size per input byte. We therefore evaluate three increasingly end-to-end attack objectives side by side.

Let h denote the current token history and let v be a candidate next token. The first objective ignores token length and selects the least probable next token as an extension of the series h :

$$\text{MinProb}(v; h) = -\log_2 p_\theta(v | h). \quad (1)$$

This is the conventional token-level worst case. It maximizes ideal coded bits per token, but not necessarily bits per source byte.

To account for variable-length tokenization, define the incremental decoded byte length

$$\Delta b(v; h) = |\text{decode}(h \parallel v)|_{\text{UTF8}} - |\text{decode}(h)|_{\text{UTF8}}. \quad (2)$$

The byte-aware greedy objective is

$$\text{SurprisalPerByte}(v; h) = \frac{-\log_2 p_\theta(v | h)}{\Delta b(v; h)}. \quad (3)$$

It directly targets the ideal entropy-coding cost per newly decoded source byte.

Finally, for a complete canonical token sequence x , let $\text{SerializeCodec}(x)$ denote the actual serialized codec output. The end-to-end objective is

$$\text{ActualRate}(x) = \frac{|\text{SerializeCodec}(x)|}{8|\text{decode}(x)|_{\text{UTF8}}}. \quad (4)$$

A realized-size beam search approximately maximizes Eq. 4 by cloning coder state for candidate continuations. This objective captures arithmetic coding and serialization effects that ideal NLL does not.

2.1 Why prediction loss is not compression loss

For a sequence of N tokens, let $|\text{decode}(x)|_{\text{UTF8}}$ denote the number of source bytes obtained after decoding the token sequence x . For example, if $\text{decode}(x) = \text{hello}$, then $|\text{decode}(x)|_{\text{UTF8}} = 5$ bytes. We define tokenizer fertility as

$$f(x) = \frac{|\text{decode}(x)|_{\text{UTF8}}}{N},$$

the average number of decoded source bytes represented by each token. If

$$L(x) = - \sum_{t=2}^N \log_2 p_\theta(x_t \mid x_{<t}) \quad (5)$$

is the sequence NLL, then the ideal predictive rate is

$$R_{\text{ideal}}(x) = \frac{L(x)}{8|\text{decode}(x)|_{\text{UTF8}}} \approx \underbrace{\frac{L(x)}{N}}_{\text{bits/token}} \underbrace{\frac{N}{|\text{decode}(x)|_{\text{UTF8}}}}_{\text{tokens/byte}} \frac{1}{8}. \quad (6)$$

Thus high bits/token can be offset by high bytes/token. Tokenization can buffer severe prediction failure when adversarial tokens represent many source bytes.

Equation 6 directly gives the compression boundary: ignoring finite-coder and metadata overhead, predictive coding compresses relative to raw bytes if and only if

$$\frac{L(x)}{N} < 8 \frac{|\text{decode}(x)|_{\text{UTF8}}}{N}. \quad (7)$$

Thus the break-even prediction cost grows linearly with tokenizer fertility. A token representing f source bytes can tolerate up to $8f$ bits of surprisal before the predictive layer expands the source.

2.2 Adversarial model and generation policies

We assume white-box access to the compression pipeline: the adversary can inspect the tokenizer, model logits, and coding algorithm when constructing an input and construct length- N adversarial inputs by autoregressive search over a candidate alphabet G using different scoring functions. To ensure that generated token sequences survive decoding and retokenization, every prefix must be satisfy, $T(\text{decode}(x_{1:t})) = x_{1:t}$ as final decoding must recover exactly the same IDs and bytes (see appendix for more infos on canonical decoding).

We compare three generation policies that we benchmark against natural text from wikipedia (text8 in UTF-8) and random string sequences as baselines: *Minimum probability* greedily maximizes Eq. 1. *Maximum surprisal/byte* greedily maximizes Eq. 3. *Realized-size beam* searches over multiple canonical continuations to approximately maximize Eq. 4.

The *one-byte constraint* restricts the candidate alphabet G to canonical tokens representing exactly one decoded byte, thereby removing variation in token fertility. Under this constraint, the multi-byte buffering is removed and MinProb and SurprisalPerByte induce the same greedy ranking.

Global mask rescoring as an optimization When evaluating occurring-token dictionaries, we construct each adversarial sequence once under the full distribution and replay identical IDs and histories under the post-hoc occurring-token mask. This isolates probability renormalization; it is not a mask-adaptive attack.

3 Experimental design

Model and codec. We chose Qwen2.5-0.5B with its $\sim 152\text{K}$ token vocabulary as a The natural-text control uses the first 100,000 text8 tokens with context length 1,000, retained context 100, KV caching, and arithmetic coding over 16 independently seeded batches.

Adversarial runs. We run three independently initialized 1,000-token adversarial sequences and report means with sample standard deviation across runs.

All completed sequences satisfy Eq. ?? and decode to on average 6,914 $\pm$ X UTF-8 bytes(for the three seeded runs with different starting token). Their post-hoc occurring-token dictionaries contain 595 $\pm$ tokens.

Exact seeds, context parameters, per-run statistics, and codec configuration are reported in the appendix. All experimental variables (sequence-length, context, codec settings, etc.) are fixed/stable/the same across all runs. We report adversarial means with sample standard deviation across the three starts. The matched text8 run is deterministic for each dictionary policy and therefore has no error bar.

Table 1: End-to-end compression under increasingly system-aware adversaries. Token-level surprisal alone is not predictive of compression failure: the relevant quantity is source-normalized rate. Values below $1\times$ indicate compression relative to raw UTF-8; values above $1\times$ indicate expansion.

Input / attack	Bits/token	Bytes/token	Ideal bits/byte	Actual bits/byte	Size ratio
<i>Baselines</i>					
Natural text (text8, UTF-8)	...	...	...	...	...
Random canonical (UTF-8)	...	...	...	...	...
<i>Full-vocabulary attacks</i>					
Min probability	...	...	...	...	...
Max surprisal/byte	...	...	...	...	...
Realized-size beam	...	...	...	...	...
<i>Tokenizer ablation</i>					
One-byte constrained	...	1.000	...	...	...

112 **Reported quantities.** For every input or attack we report the same decomposition:

$$\text{Bits/token} = \frac{L(x)}{N_{\text{pred}}}, \quad (8)$$

$$\text{Bytes/token} = \frac{|\text{decode}(x)|_{\text{UTF8}}}{N}, \quad (9)$$

$$\text{Ideal bits/byte} = \frac{L(x)}{|\text{decode}(x)|_{\text{UTF8}}}, \quad (10)$$

$$\text{Realized size ratio } R = \frac{|\text{SerializeCodec}(x)|}{|\text{decode}(x)|_{\text{UTF8}}}. \quad (11)$$

113 A realized size ratio $R < 1$ denotes compression relative to raw UTF-8, whereas $R > 1$ denotes
114 expansion.

115 4 Results

116 **Prediction failure does not imply compression failure.** The existing minimum-probability full-
117 vocabulary attack reaches 28.409 ± 0.077 ideal bits per predicted token, $5.734\times$ the matched text8
118 value of 4.954 bits/token. Because a fixed ID for the 151,643-token vocabulary requires 18 bits,
119 prediction makes this adversarial token stream substantially more expensive than fixed-width IDs.
120 Yet the sequences decode to an average 6.914 source bytes/token. Dividing model cost by this fertility
121 gives approximately 4.108 ideal bits/source byte, or 51.346% of raw UTF-8. The model therefore fails
122 badly at the token level while the composed tokenizer–predictor system still has an ideal compression
123 rate below one.

124 **Byte-aware attacks test the relevant compression objective.** Minimum probability optimizes
125 only the numerator of Eq. 6. Maximum surprisal/byte additionally penalizes tokens that obtain high
126 surprisal only by representing many bytes, while realized-size beam targets the serialized output
127 itself. Table 1 is therefore the primary comparison: it determines whether progressively more end-
128 to-end objectives increase ideal or realized bits/source byte. The corresponding full-vocabulary
129 SurprisalPerByte and realized- size beam measurements are required before making a claim that
130 MinProb is the strongest compression attack.

131 **Removing tokenization gain exposes predictive expansion.** The one-byte-ASCII ablation fixes
132 fertility at one byte/token and removes the tokenizer’s ability to compensate for poor prediction using
133 long tokens. In this setting, every accepted token contributes exactly one decoded byte, fixing fertility
134 at one byte/token. Consequently, MinProb and SurprisalPerByte become identical and the break-even
135 model cost is exactly eight bits/token.

136 The measured model floor is 135.48% of raw bytes and the arithmetic payload is 143.36%; an NLL
137 beam reaches 136.70% ideal and 144.14% arithmetic payload. Thus the predictive layer cannot

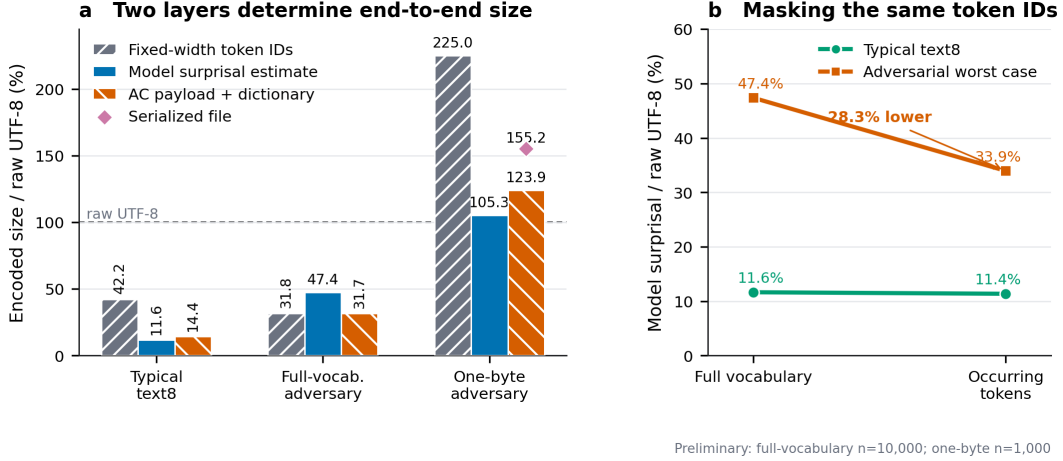


Figure 1: **Layered compression robustness and cost attribution.** (a) End-to-end size relative to raw UTF-8, separating fixed-width token IDs, the ideal model surprisal cost, the realized arithmetic-coding (AC) payload plus dictionary metadata, and the final serialized file (dot). The serialized size can be decomposed as $(S_{total} = S_{NLL} + \Delta_{AC} + S_{dict} + S_{frame})$, where (S_{NLL}) is the ideal model cost, (Δ_{AC}) is the arithmetic-coding gap above the ideal NLL, (S_{dict}) is dictionary metadata, and (S_{frame}) is the remaining container/framing overhead required to serialize the file. (b) Post-hoc restriction to occurring tokens reduces model surprisal for the same token IDs and histories, with a substantially larger reduction on the adversarial sequence than on matched text8.

beat the original 8-bit bytes once the tokenizer is prevented from absorbing model surprise through multi-byte tokens. Thus, once first-stage token compression is removed, the same kind of prediction failure that remained hidden by multi-byte Qwen tokens becomes direct end-to-end expansion. Mind that the narrow 32-token beam is an approximation of worst-case input.

Dictionary masking changes prediction cost, not the attack sequence. Post-hoc occurring-token dictionaries contain 592, 606, and 587 symbols for the three existing adversarial sequences. Replaying identical IDs and histories under these masks lowers adversarial cost from 28.409 ± 0.077 to 18.402 ± 0.833 bits/token, a 35.2% reduction, and lowers the byte-normalized ideal rate from 51.346% to 33.284%. It remains $3.806 \times$ the matched masked-text8 cost of 4.835 bits/token. Because generation is unchanged, this experiment isolates renormalization rather than adaptive robustness.

Realized overhead must be reported separately from the ideal floor. For text8, model-induced floors are 11.631% of raw UTF-8 with the full vocabulary and 11.352% with occurring tokens. Finite coding and metadata raise realized totals to 14.405% and 13.434%, corresponding to $6.942 \times$ and $7.444 \times$ compression. The occurring-token bitmap costs 10,818 bytes (2.03% of raw size), less than its coding gain at 100,000 tokens. These gaps motivate keeping ideal and actual columns separate in Table 1.

5 Systems implications and limitations

The systems lesson is to preserve token IDs as the global, queryable in-memory representation and treat predictive arithmetic coding as an optional block-level storage or transport layer.

Source-normalized accounting identifies whether fragility comes from the tokenizer, predictor, quantizer, dictionary, or container.

For high-cost inputs blocks (meaning ones with data-model mismatch), a guard can store the smaller of predictive and conventional encodings plus a selector bit, bounding size by the fallback plus one bit; its decoding granularity, latency, and throughput remain to be measured.

6 Conclusion

Robust learned compression is an end-to-end systems problem. Canonical byte-aware attacks and explicit cost attribution show that the tested adversarial sequences are 3.81–5.73 times harder than matched text8, while guarded fallback offers a principled path from diagnosis to robust operation.

References

- Grégoire Delétang, Anian Ruoss, Paul-Ambroise Duquenne, et al. Language modeling is compression. In *International Conference on Learning Representations*, 2024.
- Rico Sennrich, Barry Haddow, and Alexandra Birch. Neural machine translation of rare words with subword units. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1715–1725, Berlin, Germany, 2016. Association for Computational Linguistics. doi: 10.18653/v1/P16-1162.